5.5.27

Dhanush Kumar A - AI25BTECH11010

October 3, 2025

Question

Find the product of the matrices $\begin{pmatrix} 1 & 2 & -3 \\ 2 & 3 & 2 \\ 3 & -3 & -4 \end{pmatrix} \begin{pmatrix} -6 & 17 & 13 \\ 14 & 5 & -8 \\ -15 & 9 & -1 \end{pmatrix}$ and hence solve the system of linear equations: $x + 2y - 3z = -4$

$2x + 3y + 2z = 2$

$3x - 3y - 4z = 11$

Solution

The given matrices are

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & -3 \\ 2 & 3 & 2 \\ -3 & 2 & -4 \end{pmatrix}, \qquad \mathbf{B} = \begin{pmatrix} -6 & 17 & 13 \\ 14 & 5 & -8 \\ -15 & 9 & -1 \end{pmatrix} \tag{1}$$

The product of the matrices is computed as

$$\mathbf{AB} = \begin{pmatrix} 67 & 0 & 0 \\ 0 & 67 & 0 \\ 0 & 0 & 67 \end{pmatrix} = 67\mathbf{I}_3 \tag{2}$$

Solution

The given system of linear equations can be written in matrix form as

$$\mathbf{AX} = \mathbf{C}, \tag{3}$$

$$\mathbf{C} = \begin{pmatrix} -4 \\ 2 \\ 1 \end{pmatrix} \tag{4}$$

Since $\mathbf{AB} = 67\mathbf{I}_3$, the inverse of $\mathbf{A}$ exists and is given by

$$\mathbf{A}^{-1} = \frac{1}{67}\mathbf{B} \tag{5}$$

Solution

Multiplying both sides of $A\mathbf{X} = \mathbf{C}$ by $\mathbf{A}^{-1}$, we obtain

$$\mathbf{X} = \mathbf{A}^{-1}\mathbf{C} = \frac{1}{67}\mathbf{BC} \tag{6}$$

$$\mathbf{BC} = \begin{pmatrix} 201 \\ -134 \\ 67 \end{pmatrix} \tag{7}$$

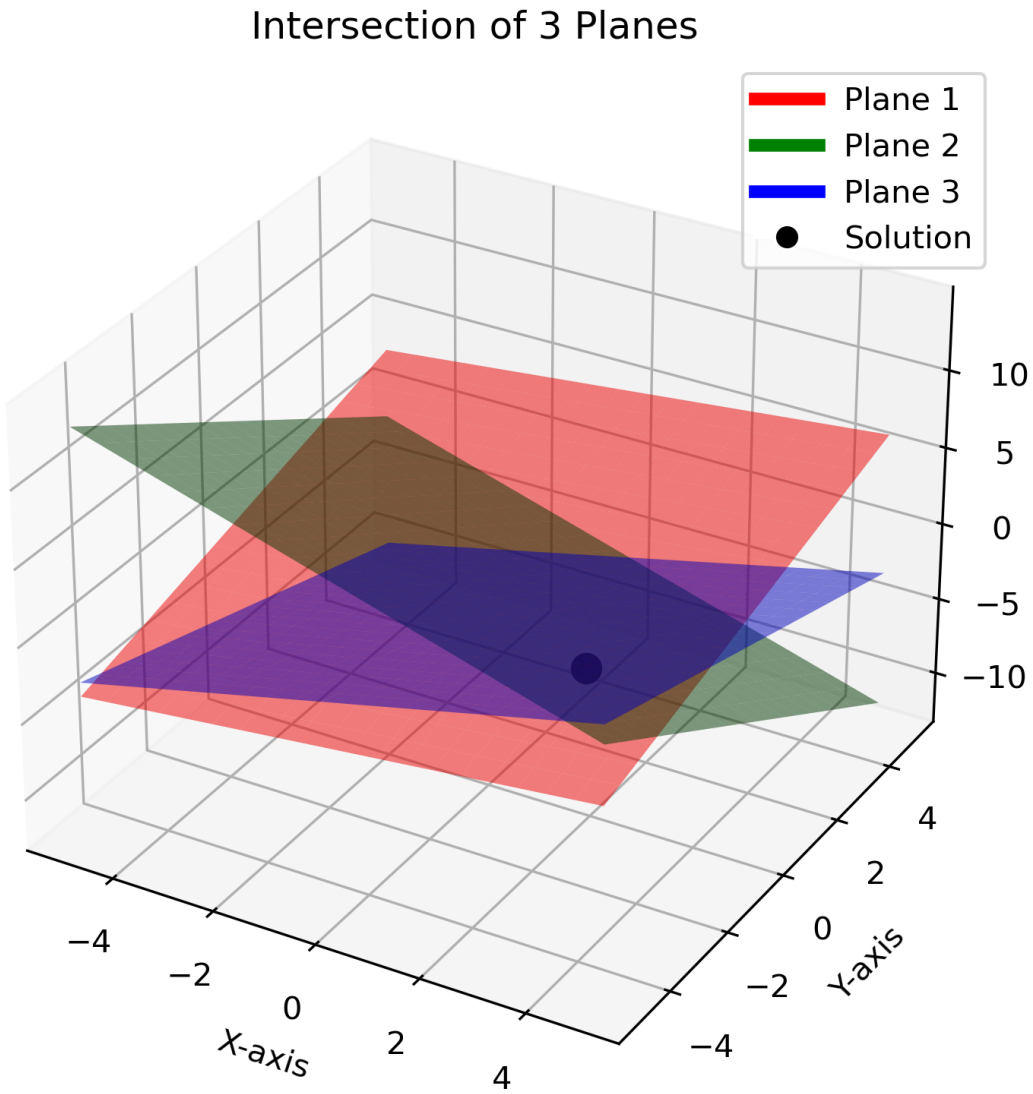
Solution

Therefore, the solution of the system is

$$\mathbf{x} = \frac{1}{67}\begin{pmatrix} 201 \\ -134 \\ 67 \end{pmatrix} = \begin{pmatrix} 3 \\ -2 \\ 1 \end{pmatrix} \tag{8}$$

$$\tag{9}$$

Plot


Intersection of 3 Planes
Plane 1
Plane 2
Plane 3
Solution
10
5
0
−5
−10
−4
−2
0
2
4
X-axis
4
2
0
−2
−4
Y-axis

Python code - To solve and plot

```
import numpy as np
import matplotlib.pyplot as plt
from mpl_toolkits.mplot3d import Axes3D
import os


# ---------- Define matrices ----------
A = np.array([[1, 2, -3],
              [2, 3, 2],
              [3, -3, -4]], dtype=float)

B = np.array([[-6, 17, 13],
              [14, 5, -8],
              [-15, 9, -1]], dtype=float)

C = np.array([-4, 2, 11], dtype=float)
```

Python code - To solve and plot

```
# ---------- Use AB = 67I3 property ----------
BC = B @ C # matrix multiplication
X = (1/67) * BC # solution vector

print("Solution in matrix form:")
print(X.reshape(-1, 1)) # column vector

# ---------- Plotting ----------
fig = plt.figure(figsize=(8, 6))
ax = fig.add_subplot(111, projection="3d")

# Create grid for planes
xx, yy = np.meshgrid(np.linspace(-5, 5, 20),
                     np.linspace(-5, 5, 20))
```

Python code - To solve and plot

```
# Plane 1: x + 2y - 3z = -4 -> z = (x + 2y + 4)/3
z1 = (xx + 2*yy + 4)/3

# Plane 2: 2x + 3y + 2z = 2 -> z = (2 - 2*xx - 3*yy)/2
z2 = (2 - 2*xx - 3*yy)/2

# Plane 3: 3x - 3y - 4z = 11 -> z = (3*xx - 3*yy - 11)/4
z3 = (3*xx - 3*yy - 11)/4

# Plot planes
ax.plot_surface(xx, yy, z1, alpha=0.5, color="red")
ax.plot_surface(xx, yy, z2, alpha=0.5, color="green")
ax.plot_surface(xx, yy, z3, alpha=0.5, color="blue")
```

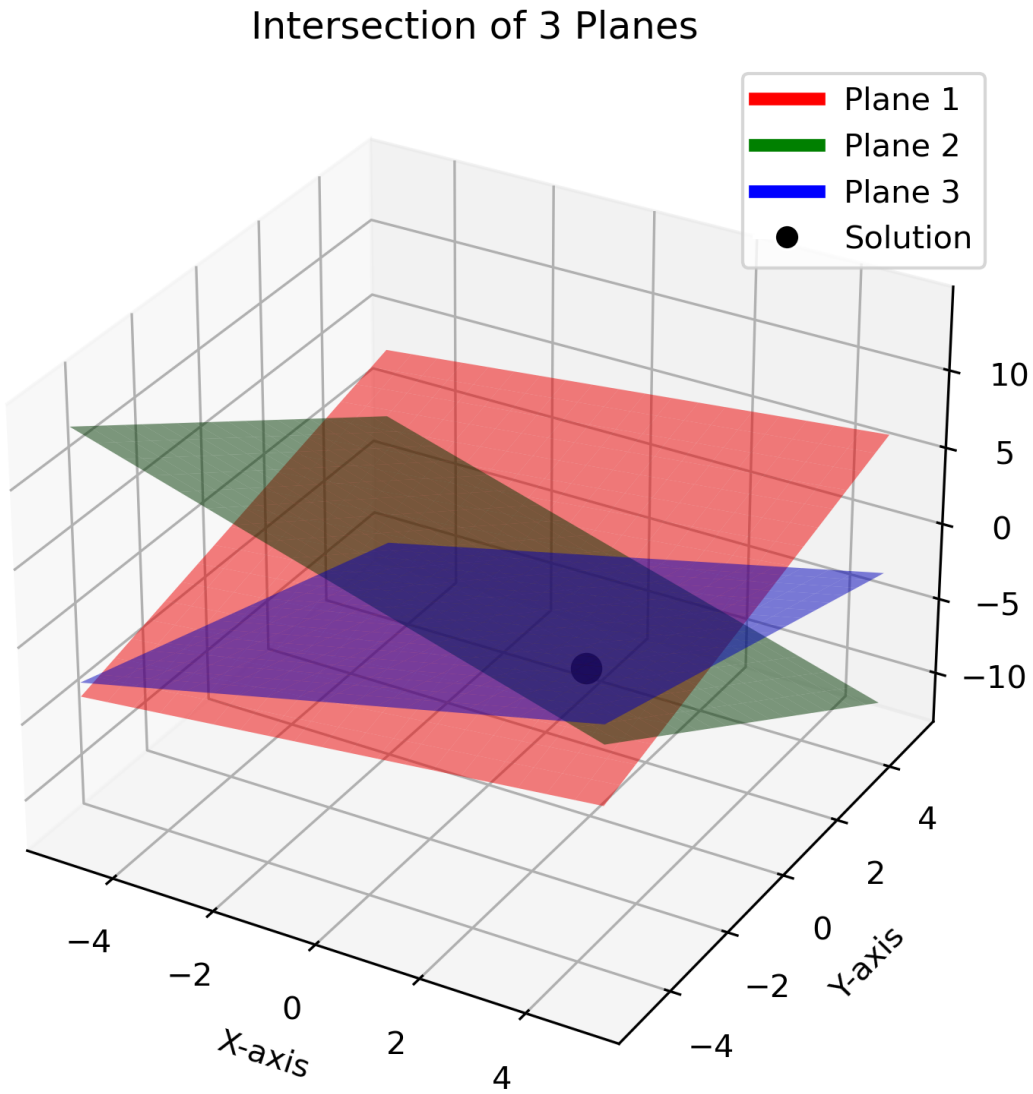
Python code - To solve and plot

```
# Plot solution point
ax.scatter(X[0], X[1], X[2], color="black", s=80, label="Solution
    (x,y,z)")

# Labels
ax.set_xlabel("X-axis")
ax.set_ylabel("Y-axis")
ax.set_zlabel("Z-axis")
ax.set_title("Intersection of 3 Planes")
```

Python code - To solve and plot

```
# Legend
custom_lines = [
    plt.Line2D([0], [0], color="red", lw=4),
    plt.Line2D([0], [0], color="green", lw=4),
    plt.Line2D([0], [0], color="blue", lw=4),
    plt.Line2D([0], [0], marker='o', color="w", markerfacecolor="
        black", markersize=8)
]
ax.legend(custom_lines, ["Plane 1", "Plane 2", "Plane 3", "
    Solution"])

# Save figure
plt.savefig("../figs/planes_solution.png", dpi=300, bbox_inches="
    tight")
plt.show()
```

Plot-Using Python



C code - Solving

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h> // <-- Needed for M_PI, cos, sin, sqrt, pow
#include "/home/dhanush-kumar-a/ee1030-2025/ai25btech11010/matgeo
    /5.5.27/codes/libs/matfun.h"

// Function to solve AX = C using inverse
void solve_system(double **A, double **B, double **C, double *X)
    {
    // Multiply A and B (3x3 * 3x3)
    double **AB = Matmul(A, B, 3, 3, 3);

    // Check determinant via AB (AB = 67I)
    double det = AB[0][0]; // should be 67
    printf("det(A) via AB: %lf\n", det);
```

C code - Solving

```c
// A inverse = (1/det) * B
double **Ainv = createMat(3, 3);
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        Ainv[i][j] = B[i][j] / det;
    }
}

// X = A^-1 * C (3x3 * 3x1)
double **XC = Matmul(Ainv, C, 3, 3, 1);
for (int i = 0; i < 3; i++) {
    X[i] = XC[i][0];
}
```

C code - Solving

```
    // Free memory
    for (int i = 0; i < 3; i++) free(AB[i]);
    free(AB);
    for (int i = 0; i < 3; i++) free(Ainv[i]);
    free(Ainv);
    for (int i = 0; i < 3; i++) free(XC[i]);
    free(XC);
}
```

Python code -Ploting the plane using c function

```
import ctypes
import numpy as np
import matplotlib.pyplot as plt
import os

# Load the C shared library
lib = ctypes.CDLL("./main.so")

# Define function prototype
lib.solve_system.argtypes = [
    ctypes.POINTER(ctypes.POINTER(ctypes.c_double)), # A
    ctypes.POINTER(ctypes.POINTER(ctypes.c_double)), # B
    ctypes.POINTER(ctypes.POINTER(ctypes.c_double)), # C
    ctypes.POINTER(ctypes.c_double) # X
]
```

Python code -Ploting the plane using c function

```
# Matrices A, B, C
A = np.array([[1, 2, -3],
              [2, 3, 2],
              [3, -3, -4]], dtype=np.double)

B = np.array([[-6, 17, 13],
              [14, 5, -8],
              [-15, 9, -1]], dtype=np.double)

C = np.array([[-4],
              [ 2],
              [11]], dtype=np.double)
```

Python code -Ploting the plane using c function

```
# Convert numpy arrays to ctypes double**
def to_c_matrix(arr):
    return (ctypes.POINTER(ctypes.c_double) * arr.shape[0])(
        *[row.ctypes.data_as(ctypes.POINTER(ctypes.c_double)) for
            row in arr]
    )

A_c = to_c_matrix(A)
B_c = to_c_matrix(B)
C_c = to_c_matrix(C)

# Output vector X
X = (ctypes.c_double * 3)()
lib.solve_system(A_c, B_c, C_c, X)
```

Python code -Ploting the plane using c function

```
# Convert solution to numpy for plotting
solution = np.array([X[i] for i in range(3)])
print(" Solution X =", solution)

# ------------------- Plotting --------------------
if not os.path.exists("figs"):
    os.makedirs("figs")

fig = plt.figure(figsize=(8,6))
ax = fig.add_subplot(111, projection="3d")

# Create a grid
xx, yy = np.meshgrid(np.linspace(-10,10,50), np.linspace
    (-10,10,50))
```

Python code -Ploting the plane using c function

```
# Plane 1: x + 2y - 3z = -4 -> z = (x + 2y + 4)/3
z1 = (xx + 2*yy + 4) / 3
ax.plot_surface(xx, yy, z1, alpha=0.5, color="cyan")

# Plane 2: 2x + 3y + 2z = 2 -> z = (2 - 2x - 3y)/2
z2 = (2 - 2*xx - 3*yy) / 2
ax.plot_surface(xx, yy, z2, alpha=0.5, color="orange")

# Plane 3: 3x - 3y - 4z = 11 -> z = (3*xx - 3*yy - 11)/4
z3 = (3*xx - 3*yy - 11) / 4
ax.plot_surface(xx, yy, z3, alpha=0.5, color="green")

# Plot the solution point
ax.scatter(solution[0], solution[1], solution[2],
           color="red", s=100, label=f"Solution {solution}")
```

Python code -Ploting the plane using c function

```
# Labels
ax.set_xlabel("X-axis")
ax.set_ylabel("Y-axis")
ax.set_zlabel("Z-axis")
ax.set_title("Solution of 3 Planes")
ax.legend()

# Save figure
plt.savefig("../figs/planes_solution1.png", dpi=300)
plt.show()
```

Plot-Using Python and C